

Week 7 Office Hour Notes

7.0 Errata

10/18/24 4:30pm. Notes have added from Week 8 office hours, including but not limited to Homework 6 Solutions and **Definition 7.3.3**, which talks about free variables.

10/22/24 4:00pm. Added **Examples 7.3.2**, **Examples 7.3.4**, and **Examples 7.3.6** to the notes during supplemental lecture.

7.1 Homework 6 Solutions

Omitted to not reveal homework solutions...

7.2 First Order Logic Language

Recall that the first order language consists of the following:

Syntax symbols $(,)$

Sentential Connective symbols $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \top, \perp, \downarrow$

Note that Enderton and the powerpoint/lectures only uses the connectives $\neg, \rightarrow$, since every other predicate connective can be derived from the others (as we'll discuss more formally in week 10), as detailed in the notes next week.

Quantifiers:

Universal quantifier $\forall$

Existential quantifier $\exists$

Note that Enderton and the power point/lectures only uses the universal quantifier $\forall$, as detailed in the notes next week, since the existential quantifier can be derived from the universal quantifier $\forall$ and negation $\neg$ (using another version of DeMorgan's Law as we'll again learn in week 10)

Variables: For each $n \in \mathbb{N}$, there is a symbol v_n that we call a "variable". Informally, these are the "terms" (as we'll define later) that we either leave "free" (as we'll also define later) or

quantify over in our formulas (as we'll define later) using $\forall, \exists$.

Function Symbols: For each $n \in \mathbb{N} \cup \{0\}$, a given language contains a (sometimes empty) set of n -ary function symbols consisting of the letter f_n containing n sorts where inputs are utilized (as we'll discuss when defining terms). Note that 0-ary function symbols (aka function symbols with no sort) are called constants.

Let's call $\mathcal{F}_n$ the set of n -ary function symbols

Relation/Predicate Symbols: Every language we assume we have equality $=$, unless otherwise specified. For $n \in \mathbb{N} \cup \{0\}$, a given language contains a (sometimes empty) set of n -ary relation symbols consisting of the letter R_n contains n sorts where inputs are utilized (as we'll discuss when defining terms). Note that 0-ary relation symbols we call "sentence symbols".

Let's call $\mathcal{R}_n$ the set of n -ary relation symbols.

Examples 7.2.1. There three main languages (though we'll certainly bring up more languages) that come to mind with this course.

1. The language of Set Theory. This language consists of only the binary predicate symbol $\in$. While in this language, there's definitely more symbols we could discuss, such as $\subset, \emptyset, \cap, \cup, \times$, etc., as well as brackets $\{, \}$, we can discuss everything using only the $\in$ symbol. For example, in the discussion forum, Nathan and Yang derived the unary defining the empty set as so:

$$\begin{aligned} x = \emptyset &\iff \forall y(\neg(y \in x)) \\ &\iff \forall y(\neg(y = y)) \end{aligned}$$

The additional relation and function symbols consist of the set $\{ \in \}$

2. The language of "Elementary" Number Theory. This language consists of the predicate symbol $<$ and the constant 0, the unary function symbol called the "successor function symbol" S , and the binary function symbols called addition $+$, multiplication $\cdot$, and exponentiation $\exp$ (which is consistent with Dr. Pomper's videos and powerpoints, but note that Enderton uses the letter E).

$$\{ <, 0, S, +, \cdot, \exp \}$$

3. The language of sentential logic. This is the language we discussed in previous weeks. It does not contain equality and for each $n \in \mathbb{N}$, there exists a sentence symbol A_n .

$$\{A_n\}_{n \in \mathbb{N}}$$

7.3 Formulas of a Language

Definition 7.3.1. We define a **term** to be a string of variables and function symbols with the following recursive properties:

Property 1. Every variable v_n for each $n \in \mathbb{N}$ is a term.

Property 2. For each $n \in \mathbb{N} \cup \{0\}$, collection of n terms $t_1, \dots, t_n$, and an n -ary function symbol f_n , we consider the string $f_n t_1 \cdots t_n$ (note that it's written in Polish notation) a term (and oftentimes, you'll see it written $f_n(t_1, \dots, t_n)$ to more clearly see what the term refers to (in a similar way that we omit parentheses in sentential logic, we don't necessarily write the terms in the technical syntax because we sometimes just want to be clear what we're referring to).

Let U_1 be the set of strings of variables and function symbols in first order logic. The set of terms is the smallest set closed under the "set builder functions" containing the base terms as variables $\{v_n\}_{n \in \mathbb{N}}$ and also constant symbols (if any) $c \in \mathcal{F}_0$ and closed under the set builder functions $\mathcal{E}_{f_n} : U_1^n \rightarrow U_1$ defined by $\mathcal{E}_{f_n}(t_1, \dots, t_n) = f_n t_1 \cdots t_n$ for each $f_n \in \mathcal{F}_n$.

This gives us the following structural induction setup:

Structural Induction on Terms. Suppose any set of terms S that has the following properties:

(i) (*Base case*) Contains all variables $\{v_n\}_{n \in \mathbb{N}}$ and also constant symbols (if any) $c \in \mathcal{F}_0$, i.e., we have $V \cup \mathcal{F}_0 \subset S$

(ii) (*Inductive Step*) If $t_1, \dots, t_n \in S$ and $f_n \in \mathcal{F}_n$, then $f_n t_1 \cdots t_n \in S$

Then S consists of all terms.

Examples 7.3.2.

(i) Note the constant symbol 0 is a term in the elementary number theory.

(ii) Note that $S0$ is a unary function symbol S , followed by a term 0 , hence $S0$ is also a term. Given a variable v_n , we note that applying S m times for an arbitrary $m \geq 1$ to v_n to get $\underbrace{S \cdots S}_{m \text{ times}} v_n$ is a term, since S is applied repeatedly to a string that is itself a term, i.e., the set

builder function $\mathcal{E}_S$ is applied m times as so:

$$\begin{aligned} \underbrace{(\mathcal{E}_S \circ \cdots \circ \mathcal{E}_S)}_{m \text{ times}}(v_n) &= \underbrace{(\mathcal{E}_S \circ \cdots \circ \mathcal{E}_S)}_{m-1 \text{ times}}(Sv_n) = \underbrace{(\mathcal{E}_S \circ \cdots \circ \mathcal{E}_S)}_{m-2 \text{ times}}(SSv_n) \\ &= \cdots = \underbrace{S \cdots S}_{m \text{ times}} v_n \end{aligned}$$

(iii) With $+$, $\cdot$, exp as binary function symbols applied to two terms give us a term, so (in Polish notation) we find

$$\begin{aligned} +S0v_n &= \mathcal{E}_+(S0, v_n), \\ \cdot SS0SSS0 &= \mathcal{E}_\cdot(SS0, SSS0), \\ \text{exp}00 &= \mathcal{E}_{\text{exp}}(0, 0) \end{aligned}$$

Definition 7.3.3. A **well-formed formula** (wff) in first order logic is a string of first order logic symbols in a given language with the following recursive properties:

Property 1. Given two terms t_1, t_2 , it consists of $= t_1 t_2$ (as written in Polish notation) and for every n -ary relation symbol R_n and collection of n terms $t_1, \dots, t_n$, it consists of the string $R_n t_1 \cdots t_n$ (again in post-fix notation). We often call this specific collection of wff's the **atomic formulas**, because these are analogous to the sentence symbols of sentential logic.

Property 2. We have closure under the predicates $\neg, \wedge, \vee$, etc. and given a wff $\varphi(x)$ that includes the use of a variable x , both $(\exists x \varphi(x))$ and $(\forall x \varphi(x))$ are also wff's.

Let U_2 be sets of strings in first order logic. The set of well formed formulas consist of the smallest set containing all strings $= t_1 t_2$ for terms t_1, t_2 and all strings $R_n t_1 \cdots t_n$ for terms $t_1, \dots, t_n$ and n -ary relation symbol $R_n \in \mathcal{R}_n$, closed under the connective set builder functions $\mathcal{E}_\neg, \mathcal{E}_\rightarrow$ (note that in Enderton/powerpoint's construction, we only consider those two connective and look at the other connectives defined in terms of those two) as defined previously in sentential logic and the quantifier set builder functions $\mathcal{E}_{\forall v_n} : U_2 \rightarrow U_2$, for each $n \in \mathbb{N}$ defined by

$$\mathcal{E}_{\forall v_n}(\phi) := (\forall v_n \phi).$$

Examples 7.3.4.

(i) In language of set theory, the string $\in v_1 v_2$ is an example of an atomic formula.

(ii) An example of a nonatomic formula is

$$\forall v_1 \forall v_2 \left(= v_1 v_2 \rightarrow \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right),$$

which can be shown to be a formula using set-builder functions as so:

$$\begin{aligned} & \forall v_1 \forall v_2 \left(= v_1 v_2 \rightarrow \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right) \\ &= \mathcal{E}_{\forall v_1} \left(\forall v_2 \left(= v_1 v_2 \rightarrow \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right) \right) \\ &= \mathcal{E}_{\forall v_1} \left(\mathcal{E}_{\forall v_2} \left(= v_1 v_2 \rightarrow \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right) \right) \\ &= \mathcal{E}_{\forall v_1} \left(\mathcal{E}_{\forall v_2} \left(\mathcal{E}_{\rightarrow} \left(= v_1 v_2, \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right) \right) \right) \\ &= \dots = \mathcal{E}_{\forall v_1} \left(\mathcal{E}_{\forall v_2} \left(\mathcal{E}_{\rightarrow} \left(= v_1 v_2, \mathcal{E}_{\forall v_3} \left(\mathcal{E}_{\rightarrow} \left(\in v_3 v_1, \in v_3 v_2 \right) \right) \right) \right) \right) \end{aligned}$$

Definition 7.3.5. Given a wff ϕ , any variable v_n that appears in the formula of ϕ (i.e., is a symbol in ϕ as a string of symbols) where no quantifier involving v_n also appears, then the variable v_n is called a **free variable**.

Some notation: We write $\phi(x_1, \dots, x_n)$ for a wff formula ϕ that has free variables $x_1, \dots, x_n$.

Equivalently, a free variable x can be defined recursively as follows:

1. If x appears in a wff of the form $= t_1 t_2$ or $R_n t_1 \dots t_n$, then x is a free variable in that formula.
2. If we have two formulas $\phi(x_1, \dots, x_n), \psi(y_1, \dots, y_m)$ with free variables $x_1, \dots, x_n$ of ϕ and $y_1, \dots, y_m$ of ψ respectively: Then
 - (i) $\neg \phi(x_1, \dots, x_n)$ still has free variables $x_1, \dots, x_n$.
 - (ii) $\phi(x_1, \dots, x_n) \rightarrow \psi(y_1, \dots, y_m)$ has free variables $z_1, \dots, z_p$, where

$$\{z_1, \dots, z_p\} := \{x_1, \dots, x_n\} \cup \{y_1, \dots, y_m\}$$

(iii) $\forall x_1 \phi(x_1, \dots, x_n)$ has free variables $x_2, \dots, x_n$ (but note it doesn't have x_1 as a free variable anymore).

Examples 7.3.6. Looking at the wff's in **Examples 7.3.4:**

(i) Note that $\in v_1 v_2$ contains the free variables v_1, v_2 , since they are variables that appear in the formula but aren't bounded by a quantifier.

(ii) However, for the formula $\forall v_1 \forall v_2 \left(= v_1 v_2 \rightarrow \forall v_3 \left(\in v_3 v_1 \rightarrow \in v_3 v_2 \right) \right)$ has no free variable since all appearances of variables v_1, v_2, v_3 are bounded by quantifiers.

7.3 Questions

To answer Evelin's question about talking about $\leq$, even though it's not explicitly defined as a symbol in the language, however, note that we can define

$$x \leq y \iff x < y \text{ or } x = y,$$

and in the language of elementary number theory, we have $=, <$ in the language, and we also have the $\vee$ connective to talk about a disjunction of statements, we we can define the binary relation $\leq$ with the wff $\varphi(x, y)$ by

$$(x < y) \vee (x = y),$$

$$\neg(x < y) \rightarrow (x = y)$$

